

Módulo 2 — Value Objects (Parte 3 de 3)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID

Caso de uso guía: **CU-CLI-001 — Alta de Cliente**

Temas de esta parte

Tema 8	Implementación: <code>FullName</code> , <code>LastName</code> , <code>Document</code>
Tema 9	Implementación: <code>Email</code> , <code>Phone</code> , <code>Address</code> , <code>BirthDate</code>
Tema 10	Actualización de <code>Customer</code> y actividades prácticas

Contexto

En este módulo se implementan los 7 Value Objects del caso de uso **CU-CLI-001 — Alta de Cliente**. Cada uno es un `sealed record` que hereda del marcador abstracto `ValueObject`, implementa un constructor privado y expone un método estático `Create()` que valida, normaliza y construye la instancia.

```
ValueObject.cs ← abstract record (marcador)
↑ hereda
sealed record FullName      sealed record LastName      sealed record Document
sealed record Email         sealed record Phone         sealed record Address
sealed record BirthDate
```

Tema 8 — Implementación: FullName, LastName, Document

8.1 — FullName

Reglas de negocio

- El nombre no puede ser nulo ni vacío.
- Debe tener al menos 2 caracteres (sin espacios).
- No puede superar 50 caracteres.
- Se almacena con `Trim()` aplicado.

```
1  // — CommerX.Domain/Customers/ValueObjects/FullName.cs —————
2  namespace CommerX.Domain.Customers.ValueObjects;
3
4  public sealed record FullName : ValueObject
5  {
6      // propiedad de solo lectura – el compilador garantiza igualdad por este valor
7      public string Value { get; }
8
9      // constructor privado – la única forma de crear instancias es via Create()
10     private FullName(string value) => Value = value;
11
12     public static FullName Create(string? value)
13     {
14         // regla 1: obligatorio
15         if (string.IsNullOrEmpty(value))
16             throw new ArgumentException("El nombre es obligatorio.", nameof(value));
17
18         var trimmed = value.Trim();
19
20         // regla 2: mínimo 2 caracteres
21         if (trimmed.Length < 2)
22             throw new ArgumentException("El nombre debe tener al menos 2 caracteres.",
23             nameof(value));
24
25         // regla 3: máximo 50 caracteres
26         if (trimmed.Length > 50)
27             throw new ArgumentException("El nombre no puede superar 50 caracteres.",
28             nameof(value));
29
30         return new FullName(trimmed);
31     }
32 }
```

8.2 — LastName

Reglas de negocio

- El apellido no puede ser nulo ni vacío.
- Debe tener al menos 2 caracteres (sin espacios).
- No puede superar 50 caracteres.
- Se almacena con `Trim()` aplicado.

```
1 // — CommerX.Domain/Customers/ValueObjects/LastName.cs
2 namespace CommerX.Domain.Customers.ValueObjects;
3
4 public sealed record LastName : ValueObject
5 {
6     public string Value { get; }
7     private LastName(string value) => Value = value;
8
9     public static LastName Create(string? value)
10    {
11        if (string.IsNullOrWhiteSpace(value))
12            throw new ArgumentException("El apellido es obligatorio.", nameof(value));
13
14        var trimmed = value.Trim();
15
16        if (trimmed.Length < 2)
17            throw new ArgumentException("El apellido debe tener al menos 2 caracteres.",
18                nameof(value));
19
20        if (trimmed.Length > 50)
21            throw new ArgumentException("El apellido no puede superar 50 caracteres.",
22                nameof(value));
23
24        return new LastName(trimmed);
25    }
26 }
```

8.3 — Document

Reglas de negocio

- El documento no puede ser nulo ni vacío.
- Debe tener entre 7 y 15 caracteres (sin espacios).
- Solo puede contener letras y números — sin guiones ni puntos.
- Se almacena en mayúsculas para comparación uniforme.

```
1 // — CommerX.Domain/Customers/ValueObjects/Document.cs
2 namespace CommerX.Domain.Customers.ValueObjects;
3
```

```
4 public sealed record Document : ValueObject
5 {
6     public string Value { get; }
7     private Document(string value) => Value = value;
8
9     public static Document Create(string? value)
10    {
11        // regla 1: obligatorio
12        if (string.IsNullOrEmpty(value))
13            throw new ArgumentException("El documento es obligatorio.", nameof(value));
14
15        var trimmed = value.Trim();
16
17        // regla 2: longitud entre 7 y 15
18        if (trimmed.Length < 7 || trimmed.Length > 15)
19            throw new ArgumentException("El documento debe tener entre 7 y 15 caracteres.",
20            nameof(value));
21
22        // regla 3: solo letras y números
23        if (!trimmed.All(char.IsLetterOrDigit))
24            throw new ArgumentException("El documento solo puede contener letras y números.",
25            nameof(value));
26
27        // normaliza a mayúsculas – "abc123" y "ABC123" producen el mismo record
28        return new Document(trimmed.ToUpper());
29    }
30 }
```

Nota sobre Document

La normalización a `ToUpper()` garantiza que dos documentos con el mismo número pero distinto formato de mayúsculas sean considerados iguales por la comparación automática del `record`. El record compara `Value` de forma exacta — sin normalizar no habría igualdad semántica.

Tema 9 — Implementación: Email, Phone, Address, BirthDate

9.1 — Email

Reglas de negocio

- El email no puede ser nulo ni vacío.
- Debe contener el carácter @.
- No puede superar 100 caracteres.
- Se almacena en minúsculas y sin espacios extremos.

```
1 // — CommerX.Domain/Customers/ValueObjects/Email.cs
2 namespace CommerX.Domain.Customers.ValueObjects;
3
4 public sealed record Email : ValueObject
5 {
6     public string Value { get; }
7     private Email(string value) => Value = value;
8
9     public static Email Create(string? value)
10    {
11        if (string.IsNullOrWhiteSpace(value))
12            throw new ArgumentException("El email es obligatorio.", nameof(value));
13        if (!value.Contains("@"))
14            throw new ArgumentException("El email debe contener @.", nameof(value));
15        if (value.Length > 100)
16            throw new ArgumentException("El email no puede superar 100 caracteres.",
17            nameof(value));
18        return new Email(value.Trim().ToLower());
19    }
20 }
```

9.2 — Phone

Reglas de negocio

- El teléfono no puede ser nulo ni vacío.
- Debe contener entre 7 y 15 dígitos (se ignoran espacios, guiones y paréntesis al contar).
- Se almacena con `Trim()` aplicado conservando el formato original.

```
1 // — CommerX.Domain/Customers/ValueObjects/Phone.cs
2 namespace CommerX.Domain.Customers.ValueObjects;
3
4 public sealed record Phone : ValueObject
5 {
```

```
6     public string Value { get; }
7     private Phone(string value) => Value = value;
8
9     public static Phone Create(string? value)
10    {
11        if (string.IsNullOrEmpty(value))
12            throw new ArgumentException("El teléfono es obligatorio.", nameof(value));
13
14        // cuenta solo los dígitos – ignora espacios, guiones, paréntesis
15        var digitCount = value.Count(char.IsDigit);
16
17        if (digitCount < 7 || digitCount > 15)
18            throw new ArgumentException("El teléfono debe tener entre 7 y 15 dígitos.",
19                nameof(value));
20
21        return new Phone(value.Trim());
22    }
23 }
```

9.3 — Address

Value Object compuesto

Address es un **Value Object compuesto**: no se define por un único valor sino por la combinación de varios campos. El record genera automáticamente la igualdad comparando **todos** sus campos: **Street**, **City**, **ZipCode** y **Country**. Si cualquiera difiere, las dos direcciones son distintas.

Reglas de negocio

- Todos los campos son obligatorios.
- Cada campo se almacena con **Trim()** aplicado.
- Dos **Address** son iguales solo si los cuatro campos coinciden exactamente.

```
1 // — CommerX.Domain/Customers/ValueObjects/Address.cs
2 namespace CommerX.Domain.Customers.ValueObjects;
3
4 public sealed record Address : ValueObject
5 {
6     // cuatro campos – el record los incluye todos en Equals() y GetHashCode()
7     public string Street { get; }
8     public string City { get; }
9     public string ZipCode { get; }
10    public string Country { get; }
11
12    private Address(string street, string city, string zipCode, string country)
13    {
14        Street = street;
15        City = city;
```

```
16         ZipCode = zipCode;
17         Country = country;
18     }
19
20     public static Address Create(
21         string? street, string? city, string? zipCode, string? country)
22     {
23         if (string.IsNullOrWhiteSpace(street))
24             throw new ArgumentException("La calle es obligatoria.", nameof(street));
25         if (string.IsNullOrWhiteSpace(city))
26             throw new ArgumentException("La ciudad es obligatoria.", nameof(city));
27         if (string.IsNullOrWhiteSpace(zipCode))
28             throw new ArgumentException("El código postal es obligatorio.", nameof(zipCode));
29         if (string.IsNullOrWhiteSpace(country))
30             throw new ArgumentException("El país es obligatorio.", nameof(country));
31
32         return new Address(street.Trim(), city.Trim(), zipCode.Trim(), country.Trim());
33     }
34 }
```

9.4 — BirthDate

Reglas de negocio

- La fecha no puede ser en el futuro.
- El cliente debe ser mayor de edad (18 años cumplidos).
- Se usa `DateOnly` — sin componente de hora.
- El cálculo de edad tiene en cuenta si el cumpleaños ya ocurrió en el año actual.

```
1  // —
2  CommerX.Domain/Customers/ValueObjects/BirthDate.cs
3
4  namespace
5  CommerX.Domain.Customers.ValueObjects;
6
7  public sealed record BirthDate : ValueObject
8  {
9      public DateOnly Value { get; }
10     private BirthDate(DateOnly value) => Value
11     = value;
12
13     public static BirthDate Create(DateOnly
14     value)
15     {
16         var today =
17         DateOnly.FromDateTime(DateTime.Today);
18
19         // regla 1: no puede ser fecha futura
20         if (value > today)
21             throw new ArgumentException("La
22             fecha de nacimiento no puede ser futura.",
```



```
nameof(value));
16
17     // regla 2: calcular edad exacta
    considerando si el cumpleaños ya pasó
18     var age = today.Year - value.Year;
19     if (new DateOnly(today.Year,
value.Month, value.Day) > today)
20         age--; // el cumpleaños aún no
    llegó este año - resta un año
21
22     if (age < 18)
23         throw new ArgumentException("El
cliente debe ser mayor de edad.",
nameof(value));
24
25         return new BirthDate(value);
26     }
27 }
```

Tema 10 — Actualización de Customer y actividades prácticas

10.1 — Entidad Customer actualizada con Value Objects

La entidad `Customer` reemplaza todos los tipos primitivos por los Value Objects implementados en los temas anteriores. Ya no realiza ninguna validación de formato: delega completamente esa responsabilidad a cada `sealed record`.

`Customer` es una entidad, por lo tanto sigue siendo una `sealed class` que hereda de `BaseEntity`. Las entidades tienen identidad (`Id`) y ciclo de vida propio — características que los records no modelan. Solo los Value Objects usan `record`.

Evolución de Customer

Módulo 1 — strings primitivos	Módulo 2 — Value Objects
<code>string FullName</code>	<code>FullName FullName</code>
<code>string LastName</code>	<code>LastName LastName</code>
<code>string Document</code>	<code>Document Document</code>
<code>string Email</code>	<code>Email Email</code>
<code>string Phone</code>	<code>Phone Phone</code>
<code>string Address</code>	<code>Address Address</code>
<code>DateTime BirthDate</code>	<code>BirthDate BirthDate</code>

```

1  // — CommerX.Domain/Customers/Entities/Customer.cs —————
2  using CommerX.Domain.Common.Entities;
3  using CommerX.Domain.Customers.ValueObjects;
4  namespace CommerX.Domain.Customers.Entities;
5
6  // Customer es una entidad: tiene Id y ciclo de vida — usa sealed class
7  public sealed class Customer : BaseEntity
8  {
9      // propiedades tipadas con Value Objects — ya no son strings primitivos
10     public FullName FullName { get; private set; } = null!;
11     public LastName LastName { get; private set; } = null!;
12     public Document Document { get; private set; } = null!;
13     public Email Email { get; private set; } = null!;
14     public Phone Phone { get; private set; } = null!;
15     public Address Address { get; private set; } = null!;
16     public BirthDate BirthDate { get; private set; } = null!;
17

```

```

18 // constructor privado vacío – requerido por EF Core para materialización
19 private Customer() { }
20
21 // método estático de fábrica – única puerta de entrada
22 public static Customer Create(
23     string    fullName,
24     string    lastName,
25     string    document,
26     string    email,
27     string    phone,
28     string    street,
29     string    city,
30     string    zipCode,
31     string    country,
32     DateOnly  birthDate)
33 {
34     // cada Create() valida y normaliza su propio dato
35     // Customer no repite ninguna validación de formato
36     return new Customer
37     {
38         FullName = FullName.Create(fullName),
39         LastName  = LastName.Create(lastName),
40         Document  = Document.Create(document),
41         Email     = Email.Create(email),
42         Phone     = Phone.Create(phone),
43         Address   = Address.Create(street, city, zipCode, country),
44         BirthDate = BirthDate.Create(birthDate)
45     };
46 }
47
48 // método de dominio: reemplaza el email – crea un nuevo record validado
49 public void UpdateEmail(string newEmail)
50     => Email = Email.Create(newEmail);
51
52 // método de dominio: reemplaza la dirección – crea un nuevo record validado
53 public void UpdateAddress(
54     string street, string city, string zipCode, string country)
55     => Address = Address.Create(street, city, zipCode, country);
56 }

```

10.2 — Estructura de archivos final del módulo

```

CommerX.Domain
+---Common
|   +---Entities
|   |   └─ BaseEntity.cs           ← abstract class (base para entidades)
|   +---ValueObjects
|   |   └─ ValueObject.cs         ← abstract record (marcador para Value
Objects)
+---Customers
+---Entities
|   └─ Customer.cs                ← sealed class : BaseEntity

```



```
+---ValueObjects
├─ FullName.cs          ← sealed record : ValueObject
├─ LastName.cs         ← sealed record : ValueObject
├─ Document.cs         ← sealed record : ValueObject
├─ Email.cs            ← sealed record : ValueObject
├─ Phone.cs            ← sealed record : ValueObject
├─ Address.cs          ← sealed record : ValueObject (compuesto)
├─ BirthDate.cs        ← sealed record : ValueObject
```

10.3 — Actividades prácticas

Caso de uso de referencia: CU-CLI-001 — Alta de Cliente

Todas las actividades están contextualizadas en la creación de un cliente con nombre, apellido, documento, email, teléfono, dirección y fecha de nacimiento.

Actividad 1 — Implementar los Value Objects

Objetivo: crear los 7 Value Objects desde cero en el proyecto.

1. Crear la carpeta `CommerX.Domain/Common/ValueObjects/`.
2. Crear `ValueObject.cs` con el marcador abstracto `public abstract record ValueObject;`.
3. Crear la carpeta `CommerX.Domain/Customers/ValueObjects/`.
4. Implementar `FullName.cs`, `LastName.cs` y `Document.cs` siguiendo el patrón del Tema 8.
5. Implementar `Email.cs`, `Phone.cs`, `Address.cs` y `BirthDate.cs` siguiendo el patrón del Tema 9.
6. Verificar que ningún Value Object tenga constructor público ni setter en sus propiedades.

Pregunta de reflexión: ¿Qué sucede si intentás hacer `new Email()` directamente? ¿Qué error produce el compilador y por qué?

Actividad 2 — Verificar igualdad por valor

Objetivo: comprobar empíricamente que los records implementan igualdad estructural.

Escribir el siguiente código en un proyecto de consola o en un test:

```
1 // verificar igualdad por valor en records
2 var email1 = Email.Create(" JUAN@MAIL.COM "); // con espacios y mayúsculas
3 var email2 = Email.Create("juan@mail.com");    // normalizado
4
5 Console.WriteLine(email1 == email2);           // ¿true o false? ¿por qué?
6 Console.WriteLine(email1.Equals(email2));      // ¿coincide con el anterior?
7 Console.WriteLine(email1);                     // ¿qué muestra ToString() del record?
8
9 var doc1 = Document.Create("abc1234");
10 var doc2 = Document.Create("ABC1234"); // mismo valor, distinto case en el input
11 Console.WriteLine(doc1 == doc2);             // ¿true o false? ¿por qué?
```

Pregunta de reflexión: ¿Por qué `email1 == email2` es `true` aunque el input era distinto? ¿Qué rol cumple la normalización en `Create()`?

Actividad 3 — Verificar que las excepciones funcionan

Objetivo: comprobar que cada Value Object rechaza valores inválidos con mensajes precisos.

1. Intentar crear un `Email` sin el carácter `@`. Observar el mensaje de la excepción.
2. Intentar crear un `Document` con menos de 7 caracteres. Observar el mensaje.
3. Intentar crear un `BirthDate` con fecha futura. Observar el mensaje.
4. Intentar crear un `BirthDate` para alguien de 15 años. Observar el mensaje.

Pregunta de reflexión: ¿En qué capa del sistema debería capturarse la excepción que lanza `BirthDate.Create()`? ¿Por qué no en la entidad?

Actividad 4 — Actualizar Customer y compilar

Objetivo: reemplazar los strings primitivos por Value Objects en `Customer`.

1. Actualizar `Customer.cs` según el Tema 10.1.
2. Compilar el proyecto. Verificar que no hay errores.
3. Crear una instancia de `Customer` con datos válidos usando `Customer.Create()`.
4. Acceder a `customer.Email.Value` y a `customer.Address.City`. Observar cómo se accede a los valores internos de los records.

Pregunta de reflexión: `Customer` usa `sealed class` y los Value Objects usan `sealed record`. ¿Qué característica de las entidades hace que no puedan ser records?

Actividad 5 — Reflexión final

Responder por escrito las siguientes preguntas:

- ¿Qué ventaja concreta tiene usar `Email email` en lugar de `string email` en la firma de un método?
- ¿Dónde vive la regla "el cliente debe ser mayor de edad"? ¿Podría vivir en otro lugar? ¿Por qué no debería?

-
- Si mañana la regla cambia a "mayores de 21 años", ¿cuántos archivos hay que modificar?
 - ¿Qué sucede si `Address.Create()` recibe `null` en el campo `city`? ¿Quién lo detecta y cuándo?
 - ¿Qué diferencia hay entre un Value Object compuesto como `Address` y uno simple como `Email`?
-

Continúa en **Módulo 3** — Excepciones personalizadas y Fail Fast:

reemplazar `ArgumentException` por excepciones de dominio propias como

`InvalidEmailException`, `InvalidDocumentException` e `InvalidAgeException`.